

Machine learning is a branch of Artificial Intelligence that focuses on developing models and algorithms that let computers learn from data without being explicitly programmed for every task. Machine Learning is mainly divided into three core types: Supervised, Unsupervised and Reinforcement Learning along with two additional types, Semi-Supervised and Self-Supervised Learning.

- **Supervised Learning:** Trains models on labeled data to predict or classify new, unseen data.
- **Unsupervised Learning:** Finds patterns or groups in unlabeled data, like clustering or dimensionality reduction.
- **Reinforcement Learning:** Learns through trial and error to maximize rewards, ideal for decision-making tasks.
- **Self supervised Learning:** Self-supervised learning is often considered as a subset of unsupervised learning, but it has grown into its own field due to its success in training large-scale models. It generates its own labels from the data, without any manual labeling.
- **Semi-Supervised Learning:** This approach combines a small amount of labeled data with a large amount of unlabeled data. It's useful when labeling data is expensive or time-consuming.

Supervised Learning Algorithms

Supervised learning algorithms are generally categorized into two main types:

- **Classification** - where the goal is to predict discrete labels or categories
- **Regression** - where the aim is to predict continuous numerical values.

1. Linear Regression

Linear regression is a type of supervised learning algorithm that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets. It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values.

This line helps us predict the dependent variable for new, unseen data. For simple linear regression (with one independent variable), the best-fit line is represented by the equation

$$y = mx + b$$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when x = 0)

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

To find the best-fit line, we use a method called Least Squares. The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.

The formula for residuals is:

$$Residual = y_i - \hat{y}_i$$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

$$sumofsqareerrors(SSE) = \sum (y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

Hypothesis function in Linear Regression

In linear regression, the hypothesis function is the equation used to make predictions about the dependent variable based on the independent variables. It represents the relationship between the input features and the target output.

For a simple case with one independent variable, the hypothesis function is:

$$h(x) = \beta_0 + \beta_1 x$$

Where:

- $h(x)$ (or $\hat{y}$) is the predicted value of the dependent variable (y).
- x is the independent variable.
- β_0 is the intercept, representing the value of y when x is 0.
- β_1 is the slope, indicating how much y changes for each unit change in x.

For **multiple linear regression** (with more than one independent variable), the hypothesis function expands to:

$$h(x_1, x_2, \dots, x_k) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k$$

Where:

- $x_1, x_2, \dots, x_k$ are the independent variables.
- β_0 is the intercept.
- $\beta_1, \beta_2, \dots, \beta_k$ are the coefficients, representing the influence of each respective independent variable on the predicted output.

Types of Linear Regression

When there is only one independent feature it is known as Simple Linear Regression or univariate Regression and when there are more than one feature it is known as Multiple Linear Regression or Multivariate Regression.

1. Simple Linear Regression

Simple Linear Regression is used when we want to predict a target value (dependent variable) using only one input feature (independent variable). It assumes a straight-line relationship between the two.

Formula:

$$\hat{y} = \theta_0 + \theta_1 x$$

Where:

- $\hat{y}$ is the predicted value
- x is the input (independent variable)
- θ_0 is the intercept (value of $\hat{y}$ when $x=0$)
- θ_1 is the slope or coefficient (how much $\hat{y}$ changes with one unit of x)

Example:

Predicting a person's salary (y) based on their years of experience (x).

2. Multiple Linear Regression

Multiple Linear Regression involves more than one independent variable and one dependent variable. The equation for multiple linear regression is:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

where:

- $\hat{y}$ is the predicted value
- $x_1, x_2, \dots, x_n$ are the independent variables
- $\theta_1, \theta_2, \dots, \theta_n$ are the coefficients (weights) corresponding to each predictor.
- θ_0 is the intercept.

The goal of the algorithm is to find the best Fit Line equation that can predict the values based on the independent variables.

In regression set of records are present with X and Y values and these values are used to learn a function so if you want to predict Y from an unknown X this learned function can be used. In regression we have to find the value of Y , So, a function is required that predicts continuous Y in the case of regression given X as independent features.

Cost function for Linear Regression

The difference between the predicted value $\hat{Y}$ and the true value Y and it is called cost function or the loss function.

In Linear Regression, the Mean Squared Error (MSE) cost function is employed, which calculates the average of the squared errors between the predicted values $\hat{y}_i$ and the actual values y_i . The purpose is to determine the optimal values for the intercept θ_1 and the coefficient of the input feature θ_2 providing the best-fit line for the given data points. The linear equation expressing this relationship is $\hat{y}_i = \theta_1 + \theta_2 x_i$.

MSE function can be calculated as:

$$\text{Cost function}(J) = \frac{1}{n} \sum_n^i (\hat{y}_i - y_i)^2$$

Utilizing the MSE function, the iterative process of gradient descent is applied to update the values of θ_1 & θ_2 . This ensures that the MSE value converges to the global minima, signifying the most accurate fit of the linear regression line to the dataset.

This process involves continuously adjusting the parameters θ_1 and θ_2 based on the gradients calculated from the MSE. The final result is a linear regression line that minimizes the overall squared differences between the predicted and actual values, providing an optimal representation of the underlying relationship in the data.

Now we have calculated loss function we need to optimize model to mitigate this error and it is done through gradient descent.

Gradient Descent for Linear Regression

Gradient descent is an optimization technique used to train a linear regression model by minimizing the prediction error. It works by starting with random model parameters and repeatedly adjusting them to reduce the difference between predicted and actual values.

2. Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression which predicts continuous values it predicts the probability that an input belongs to a specific class. It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1. It uses sigmoid function to convert inputs into a probability value between 0 and 1.

Types of Logistic Regression

Logistic regression can be classified into three main types based on the nature of the dependent variable:

- **Binomial Logistic Regression:** This type is used when the dependent variable has only two possible categories. Examples include Yes/No, Pass/Fail or 0/1. It is the most common form of logistic regression and is used for binary classification problems.
- **Multinomial Logistic Regression:** This is used when the dependent variable has three or more possible categories that are not ordered. For example, classifying animals into categories like "cat," "dog" or "sheep." It extends the binary logistic regression to handle multiple classes.
- **Ordinal Logistic Regression:** This type applies when the dependent variable has three or more categories with a natural order or ranking. Examples include ratings like

"low," "medium" and "high." It takes the order of the categories into account when modeling.

Understanding Sigmoid Function

1. The sigmoid function is an important part of logistic regression which is used to convert the raw output of the model into a probability value between 0 and 1.
2. This function takes any real number and maps it into the range 0 to 1 forming an "S" shaped curve called the sigmoid curve or logistic curve. Because probabilities must lie between 0 and 1, the sigmoid function is perfect for this purpose.
3. In logistic regression, we use a threshold value usually 0.5 to decide the class label.
 - If the sigmoid output is same or above the threshold, the input is classified as Class 1.
 - If it is below the threshold, the input is classified as Class 0.

This approach helps to transform continuous input values into meaningful class predictions.

How does Logistic Regression work?

Logistic regression model transforms the linear regression function continuous value output into categorical value output using a sigmoid function which maps any real-valued set of independent variables input into a value between 0 and 1.

Evaluation Metrics

The following metrics are commonly used:

1. **Accuracy** provides the proportion of correctly classified instances.

$$Accuracy = \frac{TruePositives + TrueNegatives}{Total}$$

2. **Precision:** focuses on the accuracy of positive predictions.

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives}$$

3. **Recall (Sensitivity or True Positive Rate):** measures the proportion of correctly predicted positive instances among all actual positive instances.

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives}$$

4. **F1 Score:** is the harmonic mean of precision and recall.

$$F1Score = 2 * \frac{Precision * Recall}{Precision + Recall}$$

5. **Area Under the Receiver Operating Characteristic Curve (AUC-ROC):** The ROC curve plots the true positive rate against the false positive rate at various thresholds. It measures the area under this curve which provides an aggregate measure of a model's performance across different classification thresholds.

6. Area Under the Precision-Recall Curve (AUC-PR): measures the area under the precision-recall curve helps in providing a summary of a model's performance across different precision-recall trade-offs.

- **Decision Tree**

A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart help to make decisions step by step where:

- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.

Decision trees are widely used due to their interpretability, flexibility and low preprocessing needs.

How Does a Decision Tree Work

A decision tree splits the dataset based on feature values to create pure subsets ideally all items in a group belong to the same class. Each leaf node of the tree corresponds to a class label and the internal nodes are feature-based decision points.

Information Gain

Information Gain tells us how useful a question (or feature) is for splitting data into groups. It measures how much the uncertainty decreases after the split. A good question will create clearer groups and the feature with the highest Information Gain is chosen to make the decision.

For example if we split a dataset of people into "Young" and "Old" based on age and all young people bought the product while all old people did not, the Information Gain would be high because the split perfectly separates the two groups with no uncertainty left

- Suppose S is a set of instances A is an attribute, S_v is the subset of S , v represents an individual value that the attribute A can take and $Values(A)$ is the set of all possible values of A then

$$Gain(S, A) = Entropy(S) - \sum_v \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

- **Entropy:** is the measure of uncertainty of a random variable it characterizes the impurity of an arbitrary collection of examples. The higher the entropy more the information content.

For example if a dataset has an equal number of "Yes" and "No" outcomes (like 3 people who bought a product and 3 who didn't), the entropy is high because it's uncertain which outcome to predict. But if all the outcomes are the same (all "Yes" or all "No") the entropy is 0 meaning there is no uncertainty left in predicting the outcome

Suppose S is a set of instances, A is an attribute, S_v is the subset of S with $A=v$ and $Values(A)$ is the set of all possible values of A , then

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \cdot Entropy(S_v)$$

Building Decision Tree using Information Gain the essentials

- Start with all training instances associated with the root node
- Use info gain to choose which attribute to label each node with
- Recursively construct each subtree on the subset of training instances that would be classified down that path in the tree.
- If all positive or all negative training instances remain, the label that node "yes" or "no" accordingly
- If no attributes remain label with a majority vote of training instances left at that node
- If no instances remain label with a majority vote of the parent's training instances.
- **K-Nearest Neighbor**

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression). Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.

K-Nearest Neighbors is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification.

How to choose the value of k for KNN Algorithm?

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right k is important for good results.
- If the data has lots of noise or outliers, using a larger k can make the predictions more stable.
- But if k is too large the model may become too simple and miss important patterns and this is called underfitting.
- So k should be picked carefully based on the data.

Statistical Methods for Selecting k

- **Cross-Validation:** is a good way to find the best value of k is by using k-fold cross-validation. This means dividing the dataset into k parts. The model is trained on some of these parts and tested on the remaining ones. This process is repeated for each part. The k value that gives the highest average accuracy during these tests is usually the best one to use.
- **Elbow Method:** Here we draw a graph showing the error rate or accuracy for different k values. As k increases the error usually drops at first. But after a certain point error stops decreasing quickly. The point where the curve changes direction and looks like an "elbow" is usually the best choice for k.
- **Odd Values for k:** It's a good idea to use an odd number for k especially in classification problems. This helps avoid ties when deciding which class is the most common among the neighbors.

Distance Metrics Used in KNN Algorithm

KNN uses distance metrics to identify nearest neighbor, these neighbors are used for classification and regression task. To identify nearest neighbor we use below distance metrics:

1. Euclidean Distance

Euclidean distance is defined as the straight-line distance between two points in a plane or space. You can think of it like the shortest path you would walk if you were to go directly from one point to another.

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^d (x_j - X_{i_j})^2}$$

2. Manhattan Distance

This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets. It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

3. Minkowski Distance

Minkowski distance is like a family of distances, which includes both Euclidean and Manhattan distances as special cases.

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{\frac{1}{p}}$$

From the formula above, when $p=2$, it becomes the same as the Euclidean distance formula and when $p=1$, it turns into the Manhattan distance formula. Minkowski distance is essentially a flexible formula that can represent either Euclidean or Manhattan distance depending on the value of p .

Working of KNN algorithm

The K-Nearest Neighbors (KNN) algorithm operates on the principle of similarity where it predicts the label or value of a new data point by considering the labels or values of its K nearest neighbors in the training dataset.

Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points Euclidean distance is widely used. Distance is calculated between data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

- When you want to classify a data point into a category like spam or not spam, the KNN algorithm looks at the K closest points in the dataset. These closest points are called neighbors. The algorithm then looks at which category the neighbors belong to and picks the one that appears the most. This is called majority voting.
- In regression, the algorithm still looks for the K closest points. But instead of voting for a class in classification, it takes the average of the values of those K neighbors. This average is the predicted value for the new point for the algorithm.

5. Naive Bayes Algorithm

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each

other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorization and sentiment analysis. It is named as "Naive" because it assumes the presence of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem.

Introduction to Bayes' Theorem

Bayes' Theorem provides a principled way to reverse conditional probabilities. It is defined as:

$$P(y | X) = \frac{P(X | y) \cdot P(y)}{P(X)}$$

Where:

- $P(y | X)$: Posterior probability, probability of class y given features X
- $P(X | y)$: Likelihood, probability of features X given class y
- $P(y)$: Prior probability of class y
- $P(X)$: Marginal likelihood or evidence

- **Random Forest (Bagging Algorithm)**

Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it an ensemble learning technique. This helps in improving accuracy and reducing errors.

Working of Random Forest Algorithm

- **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Pick Random Features:** When building each tree it doesn't look at all the features (columns) at once. It picks a few at random to decide how to split the data. This helps the trees stay different from each other.
- **Each Tree Makes a Prediction:** Every tree gives its own answer or prediction based on what it learned from its part of the data.
- **Combine the Predictions:** For **classification** we choose a category as the final answer is the one that most trees agree on i.e majority voting and for **regression** we predict a number as the final answer is the average of all the trees predictions.

Introduction to Ensemble Learning

It combines multiple simple models to create a stronger, smarter model. There are mainly two types of ensemble learning:

- Bagging: it combines multiple models trained independently.
- Boosting: it builds models sequentially each correcting the errors of the previous one.

Unsupervised Learning

Unsupervised learning are again divided into **three main categories** based on their purpose:

- Clustering
- Association Rule Mining: Association rules are a fundamental concept used to find relationships, correlations or patterns within large sets of data items.

- **Dimensionality Reduction:** Dimensionality reduction is used to simplify datasets by reducing the number of features while retaining the most important information.

1. Clustering

Clustering is an unsupervised machine learning technique that groups similar data points together into clusters based on their characteristics, without using any labeled data. The objective is to ensure that data points within the same cluster are more similar to each other than to those in different clusters, enabling the discovery of natural groupings and hidden patterns in complex datasets.

Types of Clustering

1. Hard Clustering: In hard clustering, each data point strictly belongs to exactly one cluster, no overlap is allowed. This approach assigns a clear membership, making it easier to interpret and use for definitive segmentation tasks.

- **Example:** If clustering customer data into 2 segments, each customer belongs fully to either Cluster 1 or Cluster 2 without partial memberships.
- **Use cases:** Market segmentation, customer grouping, document clustering.
- **Limitations:** Cannot represent ambiguity or overlap between groups; boundaries are crisp.

2. Soft Clustering: Soft clustering assigns each data point a probability or degree of membership to multiple clusters simultaneously, allowing data points to partially belong to several groups.

- **Example:** A data point may have a 70% membership in Cluster 1 and 30% in Cluster 2, reflecting uncertainty or overlap in group characteristics.
- **Use cases:** Situations with overlapping class boundaries, fuzzy categories like customer personas or medical diagnosis.
- **Benefits:** Captures ambiguity in data, models gradual transitions between clusters.

Types of Clustering Methods

Clustering methods can be classified on the basis of how they form clusters,

1. Centroid-based Clustering (Partitioning Methods)

Centroid-based clustering organizes data points around central prototypes called centroids, where each cluster is represented by the mean (or medoid) of its members. The number of clusters is specified in advance and the algorithm allocates points to the nearest centroid, making this technique efficient for spherical and similarly sized clusters but sensitive to outliers and initialization.

Algorithms:

- **K-Means:** Iteratively assigns points to nearest centroid and recalculates centroids to minimize intra-cluster variance.

- K-Medoids: Similar to K-means but uses actual data points (medoids) as centers, robust to outliers.

2. Density-based Clustering (Model-based Methods)

Density-based clustering defines clusters as contiguous regions of high data density separated by areas of lower density. This approach can identify clusters of arbitrary shapes, handles noise well and does not require predefining the number of clusters, though its effectiveness depends on chosen density parameters.

Algorithms:

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Groups points with sufficient neighbors; labels sparse points as noise.
- Optics (Ordering Points To Identify Clustering Structure): Extends DBSCAN to handle varying densities.

3. Connectivity-based Clustering (Hierarchical Clustering)

Connectivity-based (or hierarchical) clustering builds nested groupings of data by evaluating how data points are connected to their neighbors. It creates a dendrogram a tree-like structure that reflects relationships at various granularity levels and does not require specifying cluster numbers in advance, but can be computationally intensive.

Approaches:

- Agglomerative (Bottom-up): Start with each point as a cluster; iteratively merge closest clusters.
- Divisive (Top-down): Start with one cluster; iteratively split into smaller clusters.

4. Distribution-based Clustering

Distribution-based clustering assumes data is generated from a mixture of probability distributions, such as Gaussian distributions and assigns points to clusters based on statistical likelihood. This method supports clusters with flexible shapes and overlaps, but usually requires specifying the number of distributions.

Algorithm:

- Gaussian Mixture Model: Fits data as a weighted mixture of Gaussian distributions; assigns data points based on likelihood.

5. Fuzzy Clustering

Fuzzy clustering extends traditional methods by allowing each data point to belong to multiple clusters with varying degrees of membership. This approach captures ambiguity and soft boundaries in data and is particularly useful when the clusters overlap or boundaries are not clear-cut.

Algorithm:

- [Fuzzy C-Means](#): Similar to K-means but with fuzzy memberships updated iteratively.

Reinforcement Learning

Reinforcement learning interacts with environment and learn from them based on rewards.

Reinforcement Learning Methods

1. Model-Based Methods

These methods use a model of the environment to predict outcomes and help the agent plan actions by simulating potential result. It includes

- Markov decision processes
- Bellman equation
- Value Iteration Algorithm
- Monte Carlo Tree Search

2. Model-Free Methods

The agent learns directly from experience by interacting with the environment and adjusting its actions based on feedback.

- Q-Learning
- SARSA
- Monte Carlo Methods
- Reinforce Algorithm
- Actor-Critic Algorithm
- Asynchronous Advantage Actor Critic

Neural Networks

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.

Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

- **Neurons:** The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections:** Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases:** These parameters determine the strength and influence of connections.
- **Propagation Functions:** Mechanisms that help process and transfer data across layers of neurons.

- **Learning Rule:** The method that adjusts weights and biases over time to improve accuracy.

Types of Neural Network

Neural Networks can be classified into multiple types based on their Layers and depth activation filters, Structure, Neurons used, Neuron density, data flow, and so on. The types of Neural Networks are as follows:

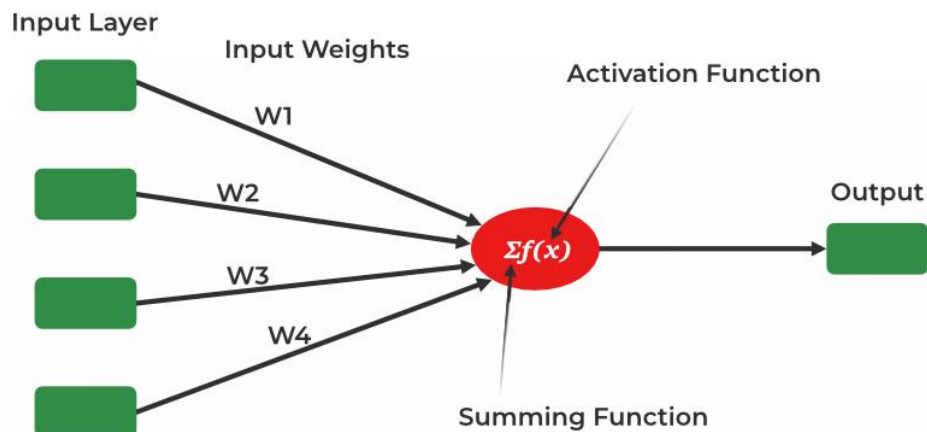
- Single-Layer Perceptron or Single Layer Neural Network
- Multi-Layer Perceptron or Multi-Layer Neural Network
- Feed Forward Neural Networks
- Convolutional Neural Networks
- Radial Basis Function Neural Networks
- Recurrent Neural Networks
- Sequence to Sequence Model
- Modular Neural Network

Single Layer Perceptron

It is one of the oldest and first introduced neural networks. It was proposed by **Frank Rosenblatt** in **1958**. Perceptron is also known as an artificial neural network. Perceptron is mainly used to compute the logic gate like **AND, OR and NOR** which has binary input and binary output.

The main functionality of the perceptron is:-

- Takes input from the input layer
- Weight them up and sum it up.
- Pass the sum to the nonlinear function to produce the output.



Here activation functions can be anything like **sigmoid**, **tanh**, **relu** based on the requirement we will be choosing the most appropriate nonlinear activation function to produce the better result.

Types of Activation Functions in Deep Learning

1. Linear Activation Function

Linear Activation Function resembles straight line define by $y=x$. No matter how many layers the neural network contains if they all use linear activation functions the output is a linear combination of the input.

- The range of the output spans from $(-\infty$ to $+\infty)$.
- Linear activation function is used at just one place i.e. output layer.
- Using linear activation across all layers makes the network's ability to learn complex patterns limited.

Linear activation functions are useful for specific tasks but must be combined with non-linear functions to enhance the neural network's learning and predictive capabilities.

2. Non-Linear Activation Functions

1. Sigmoid Function

It is characterized by 'S' shape. It is mathematically defined as $A = \frac{1}{1+e^{-x}}$. This formula ensures a smooth and continuous output that is essential for gradient-based optimization methods.

- It allows neural networks to handle and model complex patterns that linear equations cannot.
- The output ranges between 0 and 1, hence useful for binary classification.

- The function exhibits a steep gradient when x values are between -2 and 2. This sensitivity means that small changes in input x can cause significant changes in output y which is critical during the training process.

2 Tanh Activation Function

hyperbolic tangent function is a shifted version of the sigmoid, allowing it to stretch across the y -axis. It is defined as:

$$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1.$$

Alternatively, it can be expressed using the sigmoid function:

$$\tanh(x) = 2 \times \text{sigmoid}(2x) - 1$$

- **Value Range:** Outputs values from -1 to +1.
- **Non-linear:** Enables modeling of complex data patterns.
- **Use in Hidden Layers:** Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.

3. ReLU (Rectified Linear Unit) Function

It is defined by $A(x) = \max(0, x)$, this means that if the input x is positive, ReLU returns x , if the input is negative, it returns 0.

- **Value Range:** $[0, \infty)$, meaning the function only outputs non-negative values.
- **Nature:** It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
- **Advantage over other Activation:** ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.

Multi-Layer Perceptron

Multi-Layer Perceptron (MLP) consists of fully connected dense layers that transform input data from one dimension to another. It is called multi-layer because it contains an input layer, one or more hidden layers and an output layer. The purpose of an MLP is to model complex relationships between inputs and outputs. It consists of following components:

- **Input Layer:** Each neuron or node in this layer corresponds to an input feature. For instance, if you have three input features the input layer will have three neurons.
- **Hidden Layers:** MLP can have any number of hidden layers with each layer containing any number of nodes. These layers process the information received from the input layer.
- **Output Layer:** The output layer generates the final prediction or result. If there are multiple outputs, the output layer will have a corresponding number of neurons.

